Introduction to Pandas Data Structures

Introduction

Pandas is one of the most important Python libraries used in **Data Science**, **Data Analysis**, **Machine Learning**, and **Artificial Intelligence**. It provides powerful and easy-to-use data structures for storing, organizing, cleaning, and analyzing data.

The name **Pandas** is derived from the term **Panel Data**, which refers to multidimensional structured datasets commonly used in statistics and economics.

Unlike NumPy, which primarily works with homogeneous numerical arrays, Pandas can efficiently handle **heterogeneous data** (different data types) such as integers, floats, strings, dates, and Boolean values in the same dataset.

At the heart of Pandas are two fundamental data structures:

- **Series (1D)**
- **DataFrame (2D)**

Understanding these two data structures is the first step toward mastering data analysis with Pandas.

Learning Objectives

By the end of this chapter, you will be able to:

- Understand what Pandas is.
- Import the Pandas library.
- Create and work with Series.
- Create and work with DataFrames.
- Understand indexes, rows, and columns.
- Access data from Series and DataFrames.
- Understand the differences between Series, DataFrame, and NumPy arrays.

Why Do We Need Pandas?

Suppose a school stores student information.

Roll No	Marks
101	85
102	91
103	78

Managing this information using Python lists becomes difficult because:

- Data is scattered.
- Searching is slower.
- Updating records is harder.
- Statistical analysis requires more code.

Pandas solves these problems by organizing data into structured tables with powerful built-in operations.

Features of Pandas

- Easy data manipulation
- Labeled rows and columns
- Fast data analysis
- Handles missing values
- Supports multiple file formats
- Works seamlessly with NumPy
- Built-in statistical functions
- Powerful indexing and filtering

Installing Pandas

pip install pandas

Importing Pandas

import pandas as pd

Check version

import pandas as pd

print(pd.__version__)

Example Output

2.x.x

Pandas Data Structures

Pandas mainly provides two data structures:

Data Structure	Dimension	Description

Series	One-Dimensional	Stores a single column of data
DataFrame	Two-Dimensional	Stores tabular data with rows and columns

---

1. Series

A **Series** is a **one-dimensional labeled array**.

It can store:

- Integers
- Floats
- Strings
- Boolean values
- Objects

Every Series has:

- Data
- Index

Creating a Series

```
import pandas as pd


marks = pd.Series([80,90,75,95])


print(marks)
```

Output

```
0  80
```

```
1    90
2    75
3    95
dtype: int64
```

Understanding the Output

```
Index    Value

0        80
1        90
2        75
3        95
```

Each value has an associated index.

Creating Series with Custom Index

```
import pandas as pd

marks = pd.Series(
  [80,90,75],
  index=["Rahul","Priya","Aman"]
)

print(marks)
```

Output

```
Rahul    80
Priya    90
Aman     75
dtype: int64
```

Accessing Series Elements

Using Index

```
print(marks["Priya"])
```

Output

```
90
```

Using Position

```
print(marks.iloc[0])
```

Output

```
80
```

Creating Series from Dictionary

```
import pandas as pd

data = {
  "Rahul":80,
  "Priya":90,
  "Aman":75
}
```

```
marks = pd.Series(data)


print(marks)
```

Output

```
Rahul    80
Priya    90
Aman     75
dtype: int64
```

Series Properties

Index

```
print(marks.index)
```

Output

```
Index(['Rahul','Priya','Aman'], dtype='object')
```

Values

```
print(marks.values)
```

Output

```
[80 90 75]
```

Data Type

```
print(marks.dtype)
```

Output

```
int64
```

Size

print(marks.size)

Output

3

---

2. DataFrame

A **DataFrame** is a **two-dimensional labeled data structure** consisting of rows and columns.

It is the most commonly used Pandas object.

Visualization

Roll No	**Marks**
101	85
102	91

1 7
0 8
3

Each column is actually a Series.

Creating a DataFrame

```
import pandas as pd


data = {
  "Name":["Rahul","Priya","Aman"],
  "Age":[20,21,19],
  "Marks":[85,91,78]
}


df = pd.DataFrame(data)


print(df)
```

Output

```
   Name   Age   Marks
0  Rahul    20     85
1  Priya    21     91
2   Aman    19     78
```

Understanding DataFrame Structure

Columns

Name Age Marks

Rows

0 Rahul 20 85

1 Priya 21 91

2 Aman 19 78

Creating DataFrame from List of Lists

```
import pandas as pd


data = [
  ["Rahul",20,85],
  ["Priya",21,91],
  ["Aman",19,78]
]


df = pd.DataFrame(
  data,
  columns=["Name","Age","Marks"]
)


print(df)
```

Creating DataFrame from List of Dictionaries

```
import pandas as pd

data = [
  {"Name":"Rahul","Marks":85},
  {"Name":"Priya","Marks":91},
  {"Name":"Aman","Marks":78}
]

df = pd.DataFrame(data)

print(df)
```

Creating Empty DataFrame

```
import pandas as pd

df = pd.DataFrame()

print(df)
```

Output

```
Empty DataFrame
Columns: []
Index: []
```

DataFrame Attributes

Shape

print(df.shape)

Example Output

(3,3)

Meaning

- 3 Rows
- 3 Columns

Columns

print(df.columns)

Output

Index(['Name','Age','Marks'], dtype='object')

Index

print(df.index)

Output

RangeIndex(start=0, stop=3, step=1)

Data Types

print(df.dtypes)

Output

Name object

Age int64

Marks int64

dtype: object

Size

print(df.size)

Output

9

Dimensions

print(df.ndim)

Output

2

Accessing Columns

print(df["Name"])

Output

0 Rahul

1 Priya

2 Aman

Notice that a single column returned from a DataFrame is a **Series**.

Accessing Multiple Columns

print(df[["Name","Marks"]])

Output

Name Marks

0 Rahul 85

1 Priya 91

2 Aman 78

Accessing Rows Using iloc

print(df.iloc[1])

Output

Name Priya

Age 21

Marks 91

Accessing Rows Using loc

print(df.loc[0])

Output

Name Rahul

Age 20

Marks 85

Custom Index in DataFrame

```
import pandas as pd


data = {
  "Name":["Rahul","Priya","Aman"],
  "Marks":[85,91,78]
```

```
}

df = pd.DataFrame(
  data,
  index=["S1","S2","S3"]
)

print(df)
```

Output

```
    Name Marks
S1   Rahul   85
S2   Priya   91
S3    Aman   78
```

Difference Between Series and DataFrame

Feature	Series	DataFrame
Dimension	1D	2D
Structure	Single Column	Multiple Columns
Index	Yes	Yes
Rows	No	Yes

Columns	No	Yes
Can Store Multiple Data Types	Limited to one dtype per Series	Yes, each column can have a different dtype

---

Difference Between NumPy Array and Pandas DataFrame

NumPy Array	Pandas DataFrame
Uses numerical indexing	Uses labeled rows and columns
Homogeneous data	Heterogeneous data
Faster for numerical computations	Better for structured data analysis
No column names	Supports column names
Less flexible	Highly flexible

Real-Life Applications

Pandas DataFrames are used for:

- Student Management Systems
- Employee Records
- Sales Reports
- Financial Analysis
- Healthcare Data

- Customer Databases
- Survey Results
- Machine Learning Datasets
- Business Analytics
- Data Cleaning and Transformation

Best Practices

- Import Pandas as `pd`.
- Use **Series** for one-dimensional data.
- Use **DataFrame** for tabular datasets.
- Assign meaningful column names.
- Use custom indexes only when they improve readability.
- Keep data organized with consistent data types in each column.

Summary

In this chapter, you learned:

- What Pandas is.
- Why Pandas is used.
- Installing and importing Pandas.
- The two core data structures: **Series** and **DataFrame**.
- Creating Series from lists and dictionaries.
- Creating DataFrames from dictionaries, lists, and empty structures.
- Accessing rows and columns.
- Understanding DataFrame attributes.
- Differences between Series, DataFrame, and NumPy arrays.
- Real-world applications of Pandas data structures.

Key Takeaways

- **Series** is a one-dimensional labeled data structure, ideal for representing a single column of data.
- **DataFrame** is a two-dimensional tabular data structure consisting of rows and columns, making it the primary object used in Pandas.
- Every **column in a DataFrame is a Series**.

- Pandas extends the capabilities of NumPy by adding labels, heterogeneous data support, and powerful data manipulation features.
- A solid understanding of **Series** and **DataFrame** is essential before learning advanced topics such as **data selection, filtering, handling missing values, grouping, merging, and data visualization**.